

Training-Design Sensitivity of Structural-Prior Gains in Unrolled Channel Estimation for Rydberg Atomic Receivers

Abdullah Haatim

Abstract—A common way to improve a trained channel estimator is to add a structural constraint inside the network, and the improvement that follows is usually credited to that constraint. This paper asks whether that credit is earned, for magnitude-only channel estimation at a Rydberg atomic receiver.

The concern is about how such networks are trained. The training loss is an average of the error over samples covering a wide range of signal-to-noise ratios. Errors at low SNR are much larger numbers than errors at high SNR, so when they are averaged together the large numbers decide the average, and most of the training effort goes to the noisy samples. A structural constraint then helps most in the region the network was trained least, and the improvement it appears to give may be repairing a training problem rather than supplying information the network could not have learned.

We propose a control for this. Measure how the training effort is divided across the SNR range, retrain both the structured and the unstructured network with that imbalance removed, and compare them again. We apply it to our own Hankel-structured estimator, using three training designs and three random seeds for each. The structural advantage is clearly positive under the usual training, close to zero when both networks are retrained on the region where the comparison is made, and negative when the loss is corrected instead. Compared seed by seed, the change from the usual training is large and clearly separated from zero; the individual endpoint values are not, and we do not claim them.

Two further things follow. A consistent pattern within each SNR range survives where a single averaged number does not, and the sign of that averaged number depends on how the average is taken. Correcting the loss is an adaptation of a published remedy and is worth a substantial gain here, improving every one of the eighteen bin-by-seed cells — including the two the correction penalises most heavily, where the published work reports a trade rather than an improvement. A simpler loss taken in decibels needs no bins and gives no difference we can resolve. All results are simulation only.

Index Terms—Algorithm unrolling, channel estimation, evaluation methodology, phase retrieval, Rydberg atomic receiver, structural priors.

I. INTRODUCTION

MANY channel estimators are iterative. They start from a guess and repeat the same step many times, each repetition improving the estimate a little. *Unrolling* turns such an algorithm into a neural network [1], [2]: instead of repeating one fixed step T times, we lay out T copies of the step side by side as T layers, and let each layer have its own parameters that are learned from data. The shape of the computation stays

the same as the original algorithm; what changes is that the numbers inside it are now trained rather than derived.

This has become a standard method for channel estimation, including for Rydberg atomic receivers, where the receiver measures only the magnitude of the field and the phase is never observed, so that recovering the channel is a phase-retrieval problem [3], [4].

A common next step is to add an explicit structural constraint inside the unrolled loop — something that pushes the estimate towards the shape a real channel ought to have, such as a low-rank step. The improvement that follows is then reported as evidence that the constraint supplied information the network could not learn by itself.

This paper argues that such evidence is usually not decisive. The reason is a convention so common that it is rarely examined: the network is trained to minimise the average error over a set of training samples whose noise levels span a wide range.

A small example shows the problem. Suppose the error is measured as a fraction of the channel's own size, and we have just two training samples. The noisy one has error 2 and the clean one has error 0.02. Their average is 1.01. Now suppose training improves the clean sample by half, to 0.01. The average moves to 1.005 — almost nothing. But improving the noisy sample by the same fraction, to 1, moves the average to 0.51. Training follows whatever moves the average, so it works almost entirely on the noisy sample. The clean sample is barely trained, not because it does not matter, but because its numbers are small.

Now put a structural constraint into such a network. It will help most where the network is weakest, which is exactly the clean, high-SNR region. The measured improvement will be large. But that improvement is at least partly repairing a training problem rather than supplying new information, and an experiment that does not control for this cannot tell the two apart.

The imbalance itself is known, and we do not claim it. Wiesmayr *et al.* [5] state it directly for machine-learning assisted detection and decoding: a system learns one set of parameters while operating over a range of SNRs, the training data is drawn from that range, and the total loss is then dominated by the low-SNR samples, because a small relative improvement at low SNR changes the cost more than a large relative improvement at high SNR. They propose SNR deweighting as the remedy. They also observe, in their introduction, that systems are usually trained either at a single

A. Haatim is with the Department of Electrical and Electronics Engineering, Birla Institute of Technology and Science, Pilani — Pilani Campus, India (e-mail: f20220519@pilani.bits-pilani.ac.in).

SNR or on samples drawn uniformly across the target range, apparently without the effect on performance at different SNRs being questioned. That is a published statement that the convention is unexamined, which is the gap this paper works in. Separately, work on deep OFDM channel estimation has studied training on SNR-restricted samples and reports the expected trade between low-noise and high-noise accuracy [6].

What we take from that work is the mechanism and the remedy. What is not taken from it, and what this paper is about, is the consequence for *attribution*: a known training problem can create apparent evidence for a structural constraint, so an uncontrolled comparison does not establish that the constraint supplied the improvement. Wiesmayr *et al.* identify the problem and fix it; they do not ask what it does to the evidence.

The estimator studied here is our own. We built a Hankel-structured unrolled network, measured an advantage at high SNR, and believed it. We then ran a control to describe that advantage better. It removed the advantage instead, and we report that on our own method for the same reason we would report it on anyone else's.

II. SYSTEM MODEL AND ESTIMATOR

A. The Measurement

We consider an uplink system with K single-antenna users and one Rydberg atomic receiver with N vapour-cell receive elements in a uniform linear array. The users transmit P known pilot symbols, collected in $\mathbf{S} \in \mathbb{C}^{K \times P}$. A known reference field $\mathbf{B} \in \mathbb{C}^{N \times P}$ is applied at the receiver. The receiver measures only the magnitude of the total field,

$$\mathbf{Z} = |\mathbf{GS} + \mathbf{B} + \mathbf{W}|, \quad \mathbf{W}_{np} \sim \mathcal{CN}(0, \sigma^2), \quad (1)$$

where $|\cdot|$ is applied to every element and $\mathbf{G} \in \mathbb{C}^{N \times K}$ is the unknown channel. The phase of the received field is not observed.

Each user's channel is a sum of L_k plane waves across the array,

$$g_k[n] = \sum_{\ell=1}^{L_k} \alpha_{\ell,k} e^{-j(n-1)\psi_{\ell,k}}, \quad \psi_{\ell,k} = \pi \sin \theta_{\ell,k}, \quad (2)$$

with L_k selected between 3 and 7, following [7]. The path gains are scaled so that the total received power does not change with the number of paths.

This paper studies the magnitude-only Rydberg architecture. A superheterodyne Rydberg receiver recovers both amplitude and phase, and nothing here applies to it.

B. The Unrolled Estimator

The classical baseline is the expectation-maximization Gerchberg-Saxton iteration (EM-GS) of Cui *et al.* [4, Alg. 2], which adapts the classical Gerchberg-Saxton scheme [8] to this receiver. The unrolled estimator, which we call the UR-former after [3], replaces $T = 10$ iterations of EM-GS with 10 learned layers. Each layer performs a data-consistency step, then a learned filter, then a Transformer block acting across users. The architecture is not a contribution of this paper; we use it as the object of study.

C. The Structural Step Being Tested

The constraint we evaluate is a single denoising step applied to the channel estimate inside every layer. It is important to define it exactly, because it is the thing being measured.

Let $\mathcal{H}$ be the map that takes a vector $\mathbf{g} \in \mathbb{C}^N$ and writes its sliding windows as an $(N-p) \times (p+1)$ Hankel matrix, with $p = \lceil N/2 \rceil - 1$. Let $\mathcal{H}^\dagger$ average the antidiagonals, which converts such a matrix back into a vector. Let Π_r keep only the r largest singular values. The structured network replaces each column $\mathbf{g}_k$ of the estimate by

$$\mathbf{g}_k \leftarrow \mathcal{H}^\dagger \Pi_r \mathcal{H} \mathbf{g}_k, \quad r = 7. \quad (3)$$

Four properties of this step matter, and each is easy to overstate.

It is a denoising step, not a projection. Reducing the rank generally destroys the Hankel pattern, and averaging the antidiagonals to restore that pattern generally raises the rank again. One pass therefore does not land on the set of matrices that are both Hankel and low rank. Applying it twice also changes the answer a second time, so it is not idempotent, which a projection must be.

It is applied every time. There is no gate and no rank selection. Every layer applies (3) on every trial. Nothing in this paper tests a structural estimator that decides when to apply itself, which is a different method.

Its gradient is approximated. In the forward direction the network computes (3) exactly. In the backward direction it treats the step as if it were the identity. This is done because the exact derivative of a truncated SVD contains terms of the form $1/(\sigma_i^2 - \sigma_j^2)$, which become meaningless when two singular values are close together, while removing the step from the graph entirely would cut the gradient path to all earlier layers. This approximation is a design choice and is part of what is being measured.

The two networks are otherwise identical. Both have 1,586,900 parameters, the same number of layers, the same optimizer and schedule, the same data order, the same initialization and the same random seed. The step in (3) adds no trainable parameters. We write

$$\Delta_H = \text{NMSE}_{\text{no structure}} - \text{NMSE}_{\text{structure}}, \quad (4)$$

so a positive Δ_H means the structural step helps.

III. THE TRAINING PROBLEM AND THE CONTROL

A. Where the Gradient Goes

First, what the network is actually asked to do. For one training sample, the error is measured as a fraction of the channel's own size,

$$\frac{\|\hat{\mathbf{G}} - \mathbf{G}\|_F^2}{\|\mathbf{G}\|_F^2}, \quad (5)$$

where $\hat{\mathbf{G}}$ is the estimate and $\mathbf{G}$ the true channel. Dividing by $\|\mathbf{G}\|_F^2$ makes the number comparable across channels of different strengths. Training takes a batch of samples drawn from the whole SNR range, averages (5) over them, and adjusts the network's parameters so that this average goes down. As the worked example in the introduction shows, such an average

is decided by its largest terms, which are the noisy samples. We measured how strongly.

The quantity to measure is where the training effort goes. Training moves the parameters in the direction given by the gradient of the loss, so the size of the gradient produced by a group of samples says how much those samples are steering the network. We therefore split the SNR range into bins, compute the gradient from the samples of one bin alone, and ask what fraction of the total gradient size that bin accounts for. Writing g_b for the gradient computed on bin b , the *gradient share* of that bin is

$$q_b = \frac{\|g_b\|}{\sum_c \|g_c\|}. \quad (6)$$

If every bin steered the network equally, each q_b would be the same. We measure it over 24 batches per bin, at the selected checkpoint, using all trainable parameters.

Two cautions about this quantity are worth stating before we use it. First, adding up the sizes of the per-bin gradients is not the same as measuring the size of their sum, because gradients pointing in different directions partly cancel: $\sum_b \|g_b\|$ is not $\|\sum_b g_b\|$. So q_b is a normalized share of gradient sizes, not a fraction of the total gradient. Second, an equal share across bins is a reference we chose. Nothing says an optimal solution must divide its effort equally.

Measured on our estimator, 89.7% of the gradient comes from trials below 5 dB, and the per-bin share spans a factor of 30.0 across the range.

This raises a concern: the high-SNR region may be receiving too little of the training effort, so any mechanism that improves high-SNR behaviour can look as though it added information. **A skewed share does not prove this by itself.** A small gradient can also mean a region that is already fitted well. The share is a reason to check, not a verdict, and we treat it that way throughout.

B. The Control

We propose the following control, which is cheap, and which we recommend be run before a structural constraint is credited.

- 1) Measure the per-bin gradient share of the training loss.
- 2) Retrain *both* the structured and the unstructured network with the imbalance removed, matching the data budget, schedule, seed and initialization.
- 3) Measure the difference between them again.

We remove the imbalance in two different ways, because a single intervention can always be argued to have broken something else.

The first way is *focused training*. We retrain both networks using only channels with SNR between 5 and 20 dB. The high-SNR region now receives all of the training effort rather than a small part of it.

The second way is a *balanced loss*. We keep the full SNR range and instead weight each sample by a fixed factor $w(b) = c/m(b)$, where $m(b)$ is the mean per-sample normalized error in bin b , measured once before training, and c is chosen so the weights have an average of one. A bin whose errors are

large is therefore counted less, by exactly the factor by which its errors are large.

This is an adaptation of the remedy in [5], and the differences are worth stating exactly. Their weights are recomputed after every epoch from the inverse of the accumulated losses, with a stability constant and a normalization at the centre of their SNR grid. Ours are computed once, before training starts, and never change. They also note at the end of their Section 4 that one could instead deweight using the loss of a fixed baseline, such as a classical system, rather than the adaptive scheme they implement — so a fixed-weight variant is raised there, though not implemented. Ours is fixed in that sense, with one further difference: the weights come from the network's own error measured before training, not from a separate classical baseline.

The two interventions share no machinery. One changes which channels the network sees; the other changes how much each channel counts, on an unchanged training set. They are not completely independent, however: both increase the relative weight of the high-SNR region, which is where the comparison is made.

IV. NUMERICAL RESULTS

A. Experimental Setup

Unless stated otherwise we use $N = 32$ array elements, $K = 3$ users, $P = 20$ pilots, a reference-to-signal ratio of 10 dB, and 80,000 training samples over 13 epochs. Test results use 2,000 paired trials: both networks see exactly the same channels, pilots, reference field and noise in each trial.

Every experiment in this paper was *pre-registered*. Before each set of training runs, we committed a dated record containing the prediction, the threshold that would count as failure, and the rule that would decide what the result meant. Each was then scored as held or failed. This matters because with three seeds it is otherwise easy to choose an analysis after seeing the numbers.

We repeat every central experiment with three random training seeds. Two different kinds of uncertainty appear below and we keep them apart. A *test-set* interval describes variation across the 2,000 trials with the networks held fixed. A *seed* interval describes variation across the three training runs. Bootstrapping the test set does not create more training runs, so the two cannot substitute for each other. Seed intervals are two-sided 95% Student- t intervals on two degrees of freedom; with three runs, the normality they assume cannot itself be checked.

B. Where the Gain Over EM-GS Comes From

Before studying the structural step, we ask where the unrolled estimator's advantage over classical EM-GS comes from. Fig. 1 separates it.

Of the 3.345 dB total improvement, the learned per-layer filter, which has 980 parameters, contributes 0.147 dB. The attention block contributes the remaining 3.198 dB. The filter's contribution does not grow with more data; it falls slightly, from 0.193 to 0.147 dB as the training set grows from 20,000 to 80,000 samples.

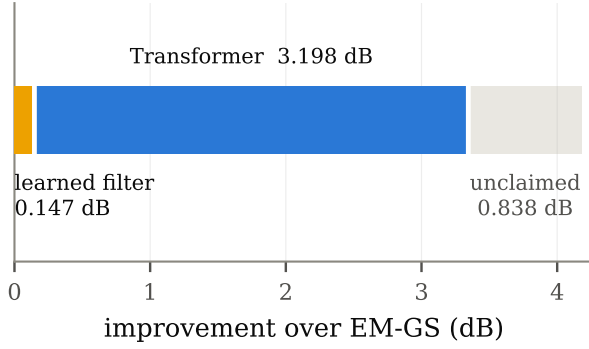


Fig. 1. Where the unrolled estimator’s gain over EM-GS comes from, at 80,000 training samples. The remainder is the gap to the genie-aided least-squares reference, which is a reference and not a bound.

This is one ablation path and not a unique division of the credit. Components can interact, and a different order of removal can give a different split. We therefore say that most of the gain along this sequence comes from the attention block, and not that the attention block is responsible for 96% of the improvement.

Unrolling also does real work on its own. Against a control that applies one Transformer block to a converged EM-GS estimate instead of unrolling, the unrolled estimator gains 0.920 dB, with a test-set interval of $[+0.888, +0.988]$, winning on 86.5% of trials.

C. Correcting the Loss

The imbalance is a property of the loss, so we fix the loss. The remedy is already known [5]; this subsection reports what it is worth in this setting. It is placed first only because the rest of the section is easier to read once the loss is corrected.

The weighting works as intended. The realized per-bin gradient share spans a factor of 1.9 after balancing against 30.0 before, and the share below 5 dB falls from 0.897 to 0.427 against an ideal of 0.5. It slightly over-corrects.

Against identically trained unbalanced networks, balancing gains +1.902, +2.242 and +2.369 dB above 5 dB on the three seeds, a mean of **+2.171** dB. The standard deviation over seeds is 0.241 dB, and the seed interval is $[+1.572, +2.771]$ dB, which excludes zero.

We had predicted, in advance, a low-SNR *cost* of -0.10 to -0.40 dB, reasoning that the lowest bin’s weight falls about eighteenfold. There was no cost.

D. Every Bin and Every Seed

The obvious objection to a reweighting is that it simply moves accuracy from one region to another: whatever the high-SNR bins gain, the down-weighted low-SNR bins must have paid. That is what one would expect, and it is what the published work on this remedy reports. Here it did not happen, and the check is exhaustive rather than pooled.

Fig. 2 plots every SNR bin for every seed. All eighteen bin-by-seed point estimates favour the balanced loss, and seventeen of them have a test-set interval that excludes zero.

The single exception is seed 1’s $[-5, 0)$ dB bin at $+0.044$ dB. The three-seed means rise from $+0.052$ dB in $[-10, -5)$ to $+3.120$ dB in $[15, 20)$.

The two lowest bins are the ones that matter for the objection, because the weighting divides their contribution by about 18 and 9. **They improve anyway.**

This differs from what the prior work reports. Wiesmayr *et al.* [5] do not report improvement in the regime their dewatering penalises; where they examine it, they report the trade. In their narrow-range experiment, dewatered training holds a small advantage at high SNR while naive training keeps an even smaller advantage at the low end of the waterfall, and their replication on a simpler system repeats the pattern: training naively across the range is worst at high SNR and best at low SNR. Their headline wide-range result is a gain at a high-SNR operating point, and no per-bin low-SNR improvement is claimed. Our measurement shows improvement everywhere, including the two most heavily penalised bins.

Three differences must be kept in view before that contrast is read as a disagreement. They are not measuring the same thing we are, not using the same mechanism, and not reporting the same size of effect.

The metric differs. They measure bit and block error rate on a coded MIMO-OFDM system. We measure normalized channel-estimation error. “Improvement in the down-weighted bins” is not the same quantity in the two papers, and a trade in one need not imply a trade in the other.

The mechanism differs. Their weights are recomputed each epoch from inverse accumulated losses, with a stability constant and a normalization at the centre of their grid. Ours are computed once before training and held fixed. Two schemes that share a motivation need not share a behaviour.

The size differs. Their dewatering gain is 0.54 dB at 1% block error rate, which they describe as modest but essentially free, since it comes from the loss function rather than from a larger receiver or more data. The gain we measure is larger. Because the metrics differ, the two numbers are not directly comparable and we do not compare them.

A reading of why our bins improve, offered as a hypothesis and not as a result: under the per-sample loss the low-SNR tail may dominate so strongly that the network is underfitted *everywhere*, in which case rebalancing stops wasting capacity rather than moving it. We have not tested that, and the measurement should not be read together with the explanation. Note also that these are eighteen individual intervals, not a simultaneous statement about all of them at once, and that a positive median in a bin does not guarantee a smaller expected squared error there.

E. The Structural Gain Under Three Training Designs

We now apply the control. Table I is the central measurement of this paper. The structural step, the architecture, the data budget, the pilot count and the test set are identical across its three rows. The only thing that changes is how training distributed its effort.

Read one row at a time, the three designs give a clearly positive value, a value near zero, and a negative value.

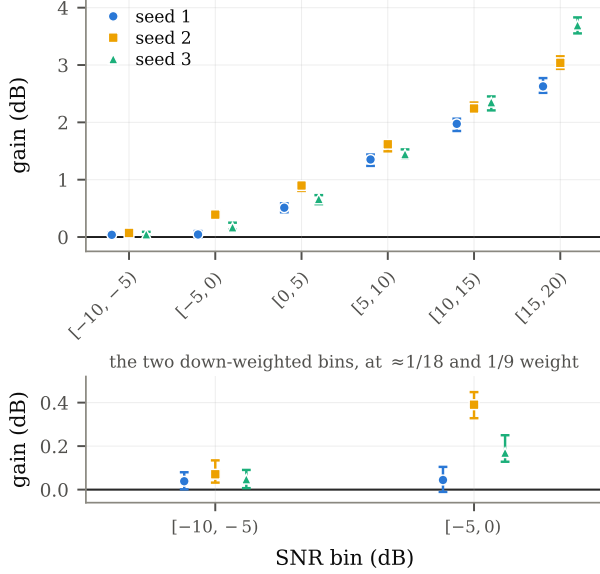


Fig. 2. Every bin, every seed. The gain from balancing the loss, with test-set intervals, for each of three training seeds. All eighteen are positive, including the two lowest bins, whose weights the balancing divides by about 18 and 9.

TABLE I

THE STRUCTURAL GAIN Δ_H ABOVE 5 dB UNDER THREE TRAINING DESIGNS, THREE SEEDS EACH. POSITIVE MEANS THE STRUCTURAL STEP HELPS. INTERVALS ARE OVER TRAINING SEEDS, NOT OVER THE TEST SET.

Training design	Δ_H (dB)	SD over seeds	95% seed interval
Conventional loss	+1.286	0.072	[+1.107, +1.465]
Focused [5, 20]	+0.020	0.083	[-0.187, +0.226]
Balanced loss	-0.177	0.150	[-0.549, +0.196]
<i>Paired change from conventional training</i>			
Focused - conventional	-1.267	0.124	[-1.574, -0.959]
Balanced - conventional	-1.463	0.179	[-1.908, -1.018]

Read as *paired differences*, the result is much sharper. The three designs use the same three seeds, so we can subtract seed by seed rather than comparing three separate averages. Doing that, the change from conventional training is -1.267 dB for focused retraining and -1.463 dB for the balanced loss. Both intervals exclude zero, with $p = 0.003$ and $p = 0.005$ from a paired t -test on two degrees of freedom.

The change is what this experiment establishes. The endpoints are not. On three seeds, the focused and balanced values are each consistent with zero, as their intervals in Table I show. We therefore do not claim that the structural gain is absent under focused training, nor that it is reliably negative under the balanced loss. What the paired comparison does support, strongly, is that the measured gain moves by more than a decibel when only the training design changes.

It is worth being clear about what this does and does not mean for the structural step. A constraint can give a real improvement at a fixed training budget, through easier optimization or through regularization, even if that improvement disappears when the training is changed. That is still useful. The distinction is between a benefit that depends on the training design and an advantage that does not, and this

TABLE II

PER-BIN STRUCTURAL GAIN Δ_H , THREE-SEED MEAN, UNDER ALL THREE TRAINING DESIGNS. A DASH MEANS THE DESIGN NEVER SAW THAT BIN. ALL EIGHTEEN UNDERLYING CELLS OF THE CONVENTIONAL DESIGN — THREE SEEDS BY SIX BINS — HAVE A TEST-SET INTERVAL EXCLUDING ZERO.

SNR bin (dB)	Conventional	Focused	Balanced
[-10, -5)	-0.120	—	-0.233
[-5, 0)	-0.400	—	-0.703
[0, 5)	-0.293	—	-0.944
[5, 10)	+0.457	-0.173	-0.692
[10, 15)	+1.403	+0.067	-0.241
[15, 20)	+2.290	+0.259	+0.567

experiment separates the two rather than denying the first.

One further design is absent by construction. We did not measure Δ_H under a loss that takes the per-sample error in decibels before averaging, because doing so would need a structured network trained with that loss and we did not train one. Three designs are reported here, not four.

F. A Single Number Hides a Sign Change

Reported as one number, Δ_H is a weighted average of two opposite effects. Table II shows the per-bin values.

The structural step *costs* accuracy at low SNR and *buys* it at high SNR. The sign changes exactly once.

Two things follow, and they behave differently.

The ordering repeats; the crossing point does not.

On every arm we have measured, every negative bin comes before every positive bin. On all three conventional seeds the sequence is exactly -, -, -, +, +, +, and all eighteen intervals exclude zero. But the bin at which the sign turns moves by up to two positions between training designs. The ordering is the property that survives; the crossing SNR is not, and neither is any single number that depends on where it falls.

The size falls under matched training, but the shape does not. Comparing the focused column against the conventional one, the two highest bins shrink by factors of about 21 and 9, while the [5, 10) bin changes sign rather than shrinking, from +0.457 to -0.173 dB. The balanced column is the same story carried further: only the highest bin stays positive, and it is down from +2.290 to +0.567 dB. In both corrected designs the top of the range keeps a positive gain and the rest of the range does not.

G. How the Average is Taken Decides the Sign

Because the per-bin contributions have opposite signs, the number one reports depends on the window the average is taken over. This includes which bins are inside it and how the trials are combined within it.

There are two natural ways to combine trials. The *paired median* takes the improvement in each trial and reports the middle value, so every trial counts equally. The *ratio of sums* adds up the squared errors of both methods and compares the totals, so a trial with a large error counts more. Across three seeds of each design:

- conventional training, restricted to $\text{SNR} \geq 5$: +1.209, +1.351, +1.298 dB by paired median; +0.723, +0.856, +0.901 dB by ratio of sums. Positive under both.
- conventional training, over the whole range: +0.129, +0.225, +0.196 dB by median; -0.214, -0.146, -0.141 dB by ratio of sums. The two *disagree in sign*.
- focused training, $\text{SNR} \geq 5$: +0.078, +0.057, -0.076 dB by median; -0.059, -0.077, -0.234 dB by ratio of sums.

The reason is measurable, and we measured it. Sorting the trials by the error of the unstructured network, the worst two deciles carry 51.2% of the total squared error and contribute -0.134 dB, while the remaining eight deciles contribute almost nothing. That is not a separate effect of trial difficulty: within a single SNR bin, the rank correlation between the baseline error and Δ_H is -0.009, -0.046 and +0.147, against -0.161 overall and +0.198 against SNR itself. In other words, the sum of squared errors is dominated by low-SNR trials, which is exactly where Table II says the structural step costs accuracy.

It is not the mechanism that produces the same symptom in the classical estimator. There, a selection rule declines to act on most trials, which pins the median at zero. The unrolled network has no such rule: the step is applied unconditionally, and it moves the estimate on every one of the 10 layers of every one of 2,000 trials on every seed, with no trial anywhere near inactive.

H. A Simpler Loss, With No Resolved Difference

Per-bin reweighting needs bin edges, a measurement pass before training to estimate the weights, and a choice of binning. Taking the per-sample error in decibels before averaging needs none of these, because the logarithm divides out the scale exactly, per sample. It is the obvious simplification, so we tested it on three seeds.

Against the reweighted network it gains +0.121 dB above 5 dB, with a standard deviation over seeds of 0.131 and a seed interval of $[-0.204, +0.446]$ dB. That interval lies inside the ± 0.50 dB equivalence margin we registered before these runs, so the pre-registered criterion for equivalence is met. Stated without reference to that margin, **the three-seed comparison simply does not resolve a difference.** Against the conventional loss, both fixes gain about the same: +2.290 dB for the log loss.

The log loss is not better balanced, either, and here our registered prediction was wrong in the direction of the error. We had predicted that the log loss would over-correct *less* than per-bin reweighting. It over-corrects more: its realized share of gradient below 5 dB is 0.317 to 0.329 against an ideal of 0.5, where per-bin reweighting reaches 0.427, and its span is 3.6 against 1.9. Dividing out the scale exactly does not equalize the gradient, because the derivative of a logarithm is itself inversely proportional to the error.

This also limits our own diagnostic. The same log-trained networks, scored under the conventional loss, put 0.906 to 0.911 of the gradient below 5 dB with a span above 40. These networks perform well and are not undertrained in any way we can detect. A skewed gradient share therefore cannot certify

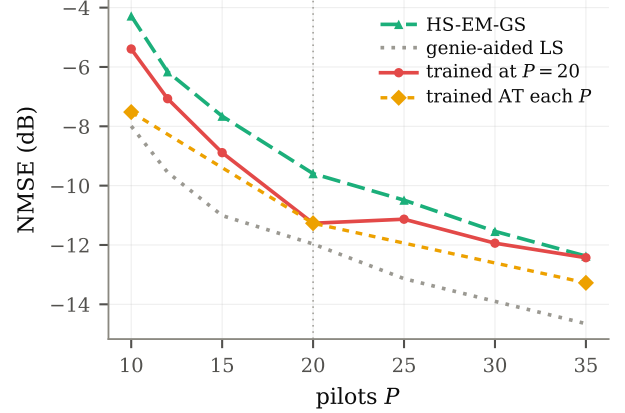


Fig. 3. Pilot efficiency against pilot-count generalization at 5 dB, on identical realizations. The dashed diamonds are separate networks trained at each P ; the solid curve is one network trained at $P = 20$.

that training was inadequate. This is the case promised earlier: the first step of our own control measures the share, and this experiment shows that the measurement is a prompt to check rather than a test that returns a verdict. We use it that way, and a reader should too.

I. Pilot Efficiency is not Pilot-Count Generalization

A pilot-count curve for a trained estimator can mean two different things. It can be one model trained at some P_0 and evaluated at each P , which measures generalization. Or it can be a separate model trained at each P , which measures efficiency. These are not close.

Trained at $P = 10$, the estimator beats the $P = 20$ model evaluated there by +2.231 dB pooled, with every bin's interval excluding zero. At $P = 35$ the corresponding margin is +0.635 dB, so the model generalizes upward in pilot count much better than downward.

At 5 dB the matched-trained estimator reaches -7.52 dB at $P = 10$, within 0.47 dB of the genie-aided reference, while the $P = 20$ model evaluated there is 2.60 dB short of it. The $P = 20$ model is also no better at $P = 25$ (-11.13 dB) than at $P = 20$ (-11.27 dB). That difference is 0.14 dB and we report it without paired uncertainty, so it shows the absence of the expected improvement rather than established harm from extra measurements.

This matters for how such curves are reported. The pilot sweep of [3] does not state whether the networks were retrained per pilot count. We report that as a fact about what the paper specifies, and as motivation for stating the distinction, not as a criticism of that work, whose convention is the ordinary one and which our own earlier curves shared.

V. LIMITATIONS AND CONCLUSION

A training loss averaged over a wide SNR range is dominated by its low-SNR tail [5]. In our setting 89.7% of the measured gradient falls below 5 dB. The consequence we draw is about evidence rather than about performance: a mechanism that improves high-SNR behaviour can appear to

add information it need not have supplied, so an uncontrolled comparison does not establish that a structural constraint supplied it.

Applied to our own Hankel-structured estimator, the control moves a +1.286 dB advantage by -1.27 dB under focused retraining and by -1.46 dB under a corrected loss, measured as paired per-seed differences with intervals excluding zero. Three designs, three seeds each, with nothing changed but how training distributed its effort. The endpoints themselves are not separated from zero on three seeds, and we do not claim them as absent or negative.

Two properties survive that a single number does not report: a per-bin sign pattern that repeats across training designs and seeds while its crossing point does not, and a dependence of the reported sign on how the average is taken. Correcting the loss is an adaptation of a published remedy and is worth +2.171 dB across three seeds here, improving all eighteen bin-by-seed point estimates. A log-domain loss that needs no bin edges gives no resolved difference from it.

Limitations. All results are simulation only, with no measured hardware and no recorded channel data. They cover the geometric ULA model of (2) at $N = 32$, $K = 3$, one data budget and the pilot counts tested.

They also cover one architecture. Everything here is measured on a single unrolled network, so we cannot separate what is a property of training design in general from what is a property of this network in particular. There is a clear way to fix that, and it is not ours: Wiesmayr *et al.* [5] repeat their whole experiment on a second, simpler system precisely to show that the effect is not an artifact of one architecture. Repeating our control on a second estimator is what the corresponding check would look like here, and we have not done it.

Three seeds is few. The paired analysis makes three go further than three separate estimates would, but it does not make three many, and every interval over seeds in this paper rests on two degrees of freedom.

We have not shown that the networks trained with the conventional loss were converged. Thirteen epochs with early stopping is a fixed budget, not a proof of adequacy. If longer training on the conventional objective removed the contrast, that would support an undertraining explanation; if it did not, the result would be about the choice of objective instead. We have not run that experiment, and the two readings are currently not separated.

The two interventions are also not fully independent. Both raise the relative weight of the high-SNR region where the comparison is made, so their agreement establishes that the measured gain depends on the training design. It does not identify which of undertraining, objective alignment or sample allocation is responsible.

Finally, the result is scoped to the structural step defined in Section II-C: a fixed rank, applied at every layer on every trial, with no rule that decides whether to apply it. This is deliberately not the same operator as the one in the classical estimator that motivated this work, which chooses its rank from held-out pilots and can decline to constrain at all. A

structural estimator that decides when to act is a different method, and nothing measured here applies to it.

Data and Pre-Registration Records

Every prediction called pre-registered above exists as a dated record written before the corresponding runs started. Each record fixes the threshold, the falsifier and the rule that would decide the outcome, and each was scored held or failed after the runs finished. The records, the scoring scripts and the per-run result files accompany this manuscript. Two of the predictions in this paper failed and are reported as failures in the text: the predicted low-SNR cost of balancing, and the prediction that the log-domain loss would correct the gradient share less than per-bin reweighting does.

REFERENCES

- [1] K. Gregor and Y. LeCun, "Learning fast approximations of sparse coding," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 399–406.
- [2] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: interpretable, efficient deep learning for signal and image processing," *IEEE Signal Process. Mag.*, vol. 38, no. 2, pp. 18–44, 2021.
- [3] J. Xiao, J. Wang, M. Zeng, H. Xu, X. Li, and A. Nallanathan, "Channel estimation for Rydberg atomic quantum receivers: unrolled phase retrieval from holographic snapshots," *IEEE Signal Process. Lett.*, vol. 33, pp. 1696–1700, 2026.
- [4] M. Cui, Q. Zeng, and K. Huang, "Towards atomic MIMO receivers," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 3, pp. 659–673, 2025.
- [5] R. Wiesmayr, G. Marti, C. Dick, H. Song, and C. Studer, "Bit error and block error rate training for ML-assisted communication," 2022, arXiv:2210.14103, v3, Mar. 2023.
- [6] A. S. M. M. Jameel, A. Malhotra, A. El Gamal, and S. Hamidi-Rad, "Deep OFDM channel estimation: capturing frequency recurrence," 2024, arXiv:2401.05436.
- [7] B. Xu, J. Zhang, Z. Chen, B. Cheng, Z. Liu, Y.-C. Wu, and B. Ai, "Channel estimation for Rydberg atomic receivers," *IEEE Wireless Commun. Lett.*, vol. 14, no. 9, pp. 2957–2961, 2025.
- [8] R. W. Gerchberg and W. O. Saxton, "A practical algorithm for the determination of phase from image and diffraction plane pictures," *Optik*, vol. 35, pp. 237–246, 1972.